

Architecture Review

DE internal Briefing: Redshift, Airflow, Monitoring & Alerts

Kevin Wei

— Agenda

01

Redshift Multi-Cluster Structure
Topology & Serverless consumers

02

Workload Management Settings
WLM queues & query prioritization

03

Airflow Query Routing
Dynamic query routing logic

04

Airflow Configuration
Variables, tags & environment configs

05

Codebase: Functions & Patterns
Query execution wrappers

06

Current Monitoring
Observability, Airflow & Redshift metrics

07

Common Alerts & Root Causes
Current alerts, example

08

Discussion
Future plan & better alerts

-
- DE internal briefing

Redshift Multi-Cluster Structure

- Sporty PROD
- All data lives physically in **one place**: the Producer cluster (bi-warehouse).
- Consumer clusters access data via Redshift Datashare — **no data duplication** required.
- Each cluster is **isolated** with independent compute, auto-scaling, and WLM configuration.
- Workload separation **prevents noisy-neighbor issues** (e.g., heavy ETL starving BI queries).
- Serverless consumers **scale to zero** when idle, ensuring highly cost-efficient irregular workloads.



Cluster Breakdown

Cluster	Type	Role	Notes
bi-warehouse	PROVISIONED	Producer · Warehouse data sync	Single physical data store; changes here affect all consumers
data-analysis	PROVISIONED	DA analytical jobs (Consumer)	Facing active issues under investigation; ongoing fixes in progress
bi-report	SERVERLESS	Ad-hoc & backfill jobs (Consumer)	Non-regular schedule; scales to zero when idle
bi-data-science	SERVERLESS	Data Science jobs (Consumer)	Serverless for cost efficiency on irregular DS workloads
bi-data-quality	SERVERLESS	OpenMetadata queries (Consumer)	System table access issues — may migrate back to bi-warehouse

- ⚠ data-analysis : recently suffer from catalog bloat and leader node CPU spike, moving some DA regular loading to bi-warehouse and bi-report
- ⚠ bi-data-quality : Serverless consumers have limited data access for system tables (ex. SVV_TABLE_INFO). may need to move back to bi-warehouse

WLM: bi-warehouse Cluster

WLM applies to Provisioned clusters only (bi-warehouse, data-analysis). Serverless clusters manage concurrency automatically.

Queue Name	Priority	Purpose	Status
▲ HIGHEST PRIORITY			
1. Monitor Queue	Highest	Redshift metrics export & OKR management queries	Active
■ NORMAL PRIORITY			
2. Default Queue	Normal	Warehouse data sync jobs	Active
3. DS Queue	Normal	Data Science queries	Plan to remove (to bi-data-science)
4. BI DAG Queries	Normal	DA analytical jobs	Plan to remove (to data-analysis)
5. BI DAG Long Queries	Normal	High-importance DA queries with extended QMR timeout	Active
6. Metabase Queries	Normal	Metabase dashboard queries with short QMR timeout	Active
▼ LOW PRIORITY			
7. OpenMetadata Queue	Low	OM test case & metadata scan queries	Queries are moved to Serverless (but might move back again)
8. Human User Queries	Low	Ad-hoc / experimental queries by DA & DS personal accounts	Plan to move to Serverless

- DE internal briefing

WLM: data-analysis Cluster & QMR Limitations

Analyzing the performance guardrails, configuration deficits, and internal Redshift behaviors on the data-analysis cluster.

Auto → Manual WLM attempt → back to Auto

- **Currently:** Auto WLM
- **In practice:** All queries route to the Default queue (only DA jobs run here now).
- **Reason to try Manual WLM:** Auto WLM does not allow setting a hard query concurrency limit.
- **Reason to fall back to Auto WLM:** queries go to planning phase first before it could be in a WLM queue.

⚠ QMR Execution Timeout: A Known Limitation

- **Behavior:** QMR execution timeout is set per queue to cancel long-running queries.
- **AWS Confirmed:** If Redshift internally rewrites a query into multiple child queries, the timeout applies at the child query level — not total execution time.
- **Result:** A query rewritten into 10 child queries each running 9 min can exceed a 10 min timeout (total 90 min) without being cancelled.

```
-- Workaround: set at session/user level instead  
SET statement_timeout TO 600000; -- 600s
```

Airflow Query Routing

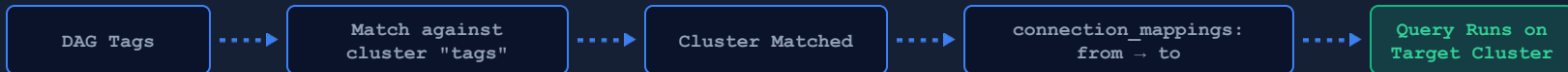
Tag-based cluster selection via `redshift_multiclustersettings`

How It Works

- DA jobs default to generic connections: `..._prod_ro` (read) and `..._prod_rw` (write).
- At runtime, routing logic reads the Airflow Variable: `redshift_multiclustersettings`.
- It checks each cluster entry's **"tags"** array against the running DAG's own tags.
- On a match, the generic connection is swapped to the cluster-specific connection defined in `connection_mappings`.
- If no tags match, the DAG stays on **bi-warehouse** (default cluster).

Tag Matching Logic

- **Cluster tag** → **match immediately**: if DAG has **cluster:XXX** tag
Example: `["cluster:bi-report"]` matches single tag
- **String Tag** → **OR Match**: DAG must have **ANY** single listed tag.
Example: `["v12", "adhoc"]` matches single tag
- **Array Tag** → **AND Match**: DAG must have **ALL** tags in sub-array.
Example: `[..., ["1_hour", "redshift"]]` requires BOTH tags to match
- **Wildcard "*"** : Connection fallback applied to any connection not explicitly mapped.



Per-Cluster Routing Configurations (current setting for Sporty PROD)

Airflow Variable: `redshift_multicluster_settings`

bi-warehouse (default)

MATCH TAGS (ANY)

```
"cluster:bi-warehouse"  
"management", "v12"
```

CONNECTION MAPPINGS

```
..._pub_prod_rw  
..._pub_prod_rw  
..._pub_prod_ro  
..._pub_prod_ro  
* ..._pub_prod_rw (fallback)
```

SYSTEM ROLE

This is effectively the default target cluster

data-analysis

MATCH TAGS (ANY)

```
"cluster:data-analysis"  
"high_importance"
```

MATCH TAGS (AND)

```
"1_hour" + "redshift"
```

CONNECTION MAPPINGS

```
..._pub_prod_ro  
..._data_analysis_ro  
* ..._data_analysis_rw
```

SYSTEM ROLE

High-importance operational pipelines and hourly Redshift loads automatically route to this cluster.

Target to handle all DA regular jobs here.

bi-report (Serverless)

MATCH TAGS (ANY)

```
"cluster:bi-report", "adhoc"  
"backfill", "hqe"
```

MATCH TAGS (AND)

```
"1_day" + "redshift"
```

CONNECTION MAPPINGS

```
..._pub_prod_rw  
..._bi_report_serverless_rw  
..._pub_prod_ro  
..._bi_report_serverless_ro  
* ..._bi_report_serverless_rw
```

SYSTEM ROLE

Ad-hoc analyst queries, heavy backfills
Some daily Redshift loadings are shifted to Serverless for now.

Airflow Configuration

Pool · Worker · Scheduler — three levers that govern DA Airflow throughput

Pool Setting

- Most jobs use `default_pool` (general-purpose, large slot count)
- Critical or high-concurrency jobs are assigned to dedicated pools with smaller slot sizes
- **Smaller pool size** = intentional concurrency cap to protect cluster and downstream systems
- To move a task to a specific pool: set `pool=<pool_name>` in the task definition

Worker Config

`PARALLELISM 128`

Global max running tasks across ALL pools and workers combined

`WORKER_CONCURRENCY 64`

Max tasks a single Celery worker picks up; multiply by worker count

`MAX_ACTIVE_TASKS_PER_DAG 4`

Default max concurrent tasks within one DAG run (overridable in code)

*Total system capacity =
`WORKER_CONCURRENCY` × workers,
capped by `PARALLELISM`.*

Scheduler Config

- `MAX_TIS_PER_QUERY = 300`
- Controls how many task instances the scheduler scans per cycle to decide what can run next
-  **If set too low:** lower priority tasks cannot be picked up in time → tasks stay stuck in "scheduled" status → eventually skipped/failed
- Setting it too high increases DB load; 300 is the current balance point

 Worker & Scheduler config defined in: `airflow-da/values-da.yaml` (data_service_deployment repo, prod branch)

- DE internal briefing

Key Utilities in s3_bucket_related.py

Central helper modules active within the data_analysis repository

Routing & Metadata

Apply multi-cluster routing

Reads

`redshift_multiclustersettings`
& swaps to mapping connection.
Centralized implementations to do
automatic routing.

Add query metadata

Prepends queries with metadata
comments. (dag_id, task_id, owner,
connection routing)
Enables query tracing in Redshift

Redshift & S3 Flows

Unload Redshift to S3

Executes Redshift UNLOAD. Defaults to
`PARALLEL OFF` now.
Can set to `PARALLEL ON` for parallel
exports. (will be default in the future)

Copy S3 data to Redshift

Runs COPY. Supports `TEMP TABLE`
mode to mitigate catalog bloat..
Prevents partial-writes and ensures
atomic swaps.

RDS MySQL & S3 Flows

Select into S3 from RDS

Executes SELECT and streams output
directly to S3.
Decouples extraction to establish S3
staging point.

Load S3 data to RDS

Get S3 objects by manifest file, and
writes into MySQL tables.
Has another batch version function to
load S3 objects one by one.

Migration Gap: Direct Calls detected across legacy DAGs

Many legacy tasks bypass these utility functions by calling raw `UNLOAD` or `COPY`. I will work on it in Q3.

Risks: ❶ Global updates (like `PARALLEL ON`) do not apply. ❷ Bypasses multiclustersettings. ❸ Loses vital metadata tracing.

Airflow & Redshift

Key dashboards, metric systems, and specific indicators for troubleshooting system degradation

Airflow Monitoring

Default Pool Slots (Airflow Alerts Dashboard)

<https://grafana-pub-prod-misc.k8s.on.sportybet2.com/d/ddvknf88xc3y8d/airflow-alerts>

Metrics tracked:

- **Running slots:** How many tasks are actively executing.
- **Scheduled slots:** How many tasks are queued, waiting for worker slots.

What to watch: When Redshift degrades, running count stays elevated and scheduled count climbs into hundreds → tasks pile up and fail.

CPU / Memory & DAG Run Delay (Airflow EKS Dashboard)

<https://grafana-pub-prod-misc.k8s.on.sportybet2.com/d/adk6avn6n2tc0aee/airflow-eks>

Metrics tracked:

- Per-DAG CPU and Memory utilization.
- **DAG run delay:** Gap between scheduled execution time and actual start.

What to watch: Sudden CPU/Mem spikes at DAG level indicate cluster trouble or bad code. DAG run delay is an early warning before pools fill.

Redshift Monitoring

AWS Console – Cluster Metrics

- **CPU Utilization & Disk Usage %**
- Read / Write Throughput
- Database Connections

AWS Console – Query Monitoring

- **Queued vs. Running query count**
- **Duration identification:** check long-running queries.

Grafana – Redshift Cluster Dashboard

<https://grafana-infra.k8s.on.sportybet2.com/d/redshift-cluster/redshift-cluster>

- Unified dashboard displaying all cluster-level infrastructure metrics in one view.
- Cross-reference directly with Airflow panels during system outages and incidents.

Common Causes of Redshift Performance Degradation

Why does the cluster suddenly get slow and tasks get timeout? core patterns that disrupt operations.

Continuous Backfill

Symptoms:

Redshift has long queries, Airflow running slots stays high, other jobs slow down.

Root Cause:

New job filling in historical data, generating heavy queries unplanned in normal capacity.

Mitigation:

- Throttle concurrency (`max_active_runs` or dedicated pool)
- Schedule backfills during off-peak hours
- Route to `bi-report` using `"backfill"` tag

High Concurrency Job

Symptoms:

Sudden jump in Redshift connections/WLM queue depth, Airflow scheduled slots spike.

Root Cause:

New job submits many parallel queries simultaneously, overwhelming the cluster.

Mitigation:

- Cap concurrency via `max_active_tasks`
- Assign to a dedicated pool with tight slot limits
- Review query patterns — batch or serialize tasks

Ad-hoc Queries

Target Accounts:

Personal accounts or team access like `da_trading`

Symptoms:

Cluster loading spikes, other jobs slow down or take longer than usual.

Root Cause:

Exploratory queries run manually on shared `bi-warehouse` impacting prod jobs.

Mitigation Plan:

Move these ad-hoc queries to **Serverless** to isolate production. (Isaac is working on)

 **Investigation Flow:** Airflow Alerts dashboard (check scheduled slot count spike) → AWS Console Query Monitoring (identify offending query) → Cross-reference with DAG owner via query metadata.

- DE internal briefing

Possible Causes: Airflow Jobs Skipped / Failed

Reference — causes can overlap and trigger each other simultaneously

● Redshift

1. Redshift internal error or service issue
2. Heavy long-running query consuming cluster resources
3. High concurrency of short queries exhausting cluster
4. Leader Node CPU spike slowing query planning

● Airflow

1. `MAX_TIS_PER_QUERY` too low — scheduler misses low-priority tasks
2. New DAG with high concurrency and low `max_active_tasks` — starves other DAGs
3. DAG with low `priority_weight` + short timeout — never picked up in time

● Connection / Auth

1. Target DB (Redshift, MySQL, etc.) has an outage or is under maintenance
2. Credential or permission changed by Github workflow
3. Network issue between Airflow workers and the target cluster

● Code

1. Bug introduced in shared general functions
2. Query Routing mechanism is changed and breaks connection mapping
3. DAG code change with unintended side effects on shared resources

⚠ **These causes are not mutually exclusive.**

A single event (e.g., new high-concurrency DAG deployed) can simultaneously trigger **Column 2** → **Column 1** → **Column 3** in a cascade.

How do we build better alerting and faster diagnosis as a team?

"Current alerts tell us something went wrong. They don't always tell us why – or which layer to look at first."

Example: Alert fires for multiple Airflow jobs skipped or failed

● Redshift performance

● Airflow DAGs / config

● Connection issue

● Codebase change

Causes can overlap and trigger each other. Multiple may be true at once.

💬 Some questions:

- When an alert fires, what do you check first and why?
- Do we have enough visibility to quickly distinguish which layer is the problem?
- Should we redesign alerts to be more specific per cause type?
- How do we prevent a new DAG / adhoc queries from silently impacting everyone else?
- What would a shared on-call triage checklist look like?

data-analysis Cluster: Ongoing Issues

3 active problems — status and mitigations in progress

● Done changes for most jobs — showing results

Catalog Bloat — Table Count Growing

What's happening:

Too many pipelines leave behind permanent staging tables via CTAS and repeated DDL. Large catalog size degrades planning and increases leader node overhead.

Root Cause:

Staging tables designed as permanent rather than session-scoped temporaries.

Fix Applied:

Migrating DAGs to **TEMP TABLE**. Ongoing, with already observed improvement in table count growth rate.

● In progress - showing results & monitor closely

Leader Node CPU Spike Under Load

What's happening:

Moderate concurrency spikes Leader Node CPU, slowing planning and compiling for all queries.

Suspected Drivers & Fixes:

- **UNLOAD PARALLEL OFF:** Outlier queries serialize through leader. Migrating to **PARALLEL ON**.
- **Concentrated Schedule:** Bursts in first 30 mins. Spreading jobs to second half of hour.

● Open — AWS Case Active

Cluster Freeze — Query Execution Blocked

What's happening:

Intermittent incidents where the cluster becomes completely unresponsive. No queries can be submitted or executed, requiring active recovery intervention.

Current Status:

- AWS Support case is actively open.
- Root cause not yet identified.
- Monitoring closely for recurrence to gather diagnostic payload.

Note: All three issues are specific to the data-analysis (Provisioned) cluster. bi-warehouse is not affected.